

Objetivo, alcance y preparación

Control

Versión 2.0 | Corte documental: 29/09/2026. Documento técnico para revisión académica. Implementado significa observado en código; no equivale a validación productiva. Las iteraciones son planes de trabajo, no actas ejecutadas.

Objetivo

Asegurar que cada identidad solo opere sobre recursos autorizados.

Situación observada

Registro, login, revocación, permisos y propiedad implementados; endurecimiento productivo pendiente.

Dependencias

I01 y catálogo de permisos revisado.

Historias de usuario

ID	Historia
HU-03	Como usuario quiero iniciar sesión y ver el panel de mi rol.
HU-04	Como administrador quiero suspender una cuenta sin permitir reutilizar sus tokens.

Trazabilidad y antecedentes

O3; RF-28, RF-33; RNF-08, RNF-09. A09, A12. Las referencias [Axx] remiten a las fichas del documento 01 y al catálogo trazabilidad/v2/articulos.csv. Páginas citadas: numeración física del PDF, salvo indicación expresa. Los originales se conservan en fuentes/articulos/Axx.pdf; SHA-256 en fuentes/articulos/manifest.json.

Incluye

api_tokens guarda SHA-256 del token y expiración; users usa password hashado e is_active. No existe tenant_id: aislamiento actual por propiedad. Multitenencia futura requiere diseño y migración específicos.

No incluye / riesgo principal

Sin recuperación de contraseña ni MFA observados. No confundir ocultar botones con autorización real.

Entrada a desarrollo

Product Owner aprueba alcance y criterio; equipo dispone de entorno QA, datos sintéticos y dependencias. Definir capacidad del sprint antes de comprometer fechas. Si esta iteración es propuesta, la documentación no significa aprobación de implementación.

Iteración 2: Autenticación, roles y aislamiento de datos

Implementación por capas

Frontend

Auth.jsx maneja registro e inicio; marketplace-api.js elimina sesión ante 401; Marketplace.jsx confirma actor mediante /state antes de mostrar datos. Probar cambio de cuenta con respuesta anterior demorada.

Backend Laravel

AuthController, ApiToken y Permissions son autoridad. Registro público solo client/technician; admin se crea con marketplace:admin. profile ignora role y userId arbitrarios. Suspensión revoca tokens; no suspender administradores ni cuentas con trabajo activo.

Python

No recibe contraseñas, Bearer tokens ni direcciones para ranking. Su ruta debe estar restringida a red interna.

Base de datos y contrato

api_tokens guarda SHA-256 del token y expiración; users usa password hashado e is_active. No existe tenant_id: aislamiento actual por propiedad. Multitenencia futura requiere diseño y migración específicos.

Secuencia para desarrollar

1. Reproducir situación actual y anotar commit. 2. Escribir caso negativo o límite que falle. 3. Implementar cambio mínimo en capa responsable. 4. Verificar respuesta y persistencia. 5. Refactorizar sin cambiar contrato. 6. Ejecutar regresión y revisar con otro integrante. 7. Actualizar evidencia y desplegar QA.

Errores y seguridad

La API vuelve a validar permisos y datos aunque la UI bloquee una acción. No usar localStorage como control real. Mantener mensajes comprensibles, datos de prueba sintéticos y secretos fuera de capturas/logs. No registrar un cambio como completo si solo existe en el frontend.

Iteración 2: Autenticación, roles y aislamiento de datos

Pruebas, entrega y finalización

Casos de aceptación

ID	Procedimiento	Resultado esperado
I02-01	Cliente B intenta cancelar solicitud de A	403 y ningún cambio de estado
I02-02	Registro público envía role=admin	422; no crear usuario
I02-03	Suspender, reactivar y reutilizar token anterior	401; debe iniciar sesión otra vez
I02-04	Técnico no asignado consulta solicitud ofrecida	Dirección no revelada; no puede iniciar trabajo

Ejecución TDD

RED: ejecutar el caso que demuestra la falta o fallo, conservar resultado. GREEN: implementar y repetir. REFACTOR: limpiar código y ejecutar regresión pertinente. Para funciones ya implementadas, registrar prueba de caracterización y no inventar una fase RED anterior.

Despliegue de la iteración

Migrar acceso en QA; crear administrador por comando interactivo sin contraseña predeterminada; smoke de tres roles. Tokens y secretos fuera de Git.

Criterios de salida

Historias aceptadas, permisos negativos aprobados, contrato y diccionario actualizados, pruebas pertinentes sin fallos y QA verificable. Guardar build/commit, entorno, caso y observado. Riesgos pendientes con responsable por asignar; ninguna firma ni aceptación simulada.

Carpeta de evidencia

evidencias/v2/I02/ contiene README de estado. En la ejecución se añadirán logs, casos, incidentes, retest, acta de revisión y registro de despliegue. Actualmente el estado es PLANIFICADO / NO EJECUTADO como iteración. No confundir pruebas previas con cierre de sprint.